FastLens: FastLanes for High-Throughput AI Image Pipelines

Abstract

Modern image AI systems can spend substantial time preparing and moving input data to GPUs. Conventional pipelines read, decode, and transfer image data before applying model-specific selection, causing unnecessary data movement and leaving expensive GPUs waiting for inputs. We present ***FastLens***, the first image AI input system to unify storage-level selection, compressed transfer, and GPU-native processing through a new compressed image format and end-to-end pipeline. FastLens addresses three key challenges. First, fine-grained selection can sacrifice data locality and batch efficiency. We design a cross-image block-major layout with dependency-aware projection pushdown for efficient selective and batched access. Second, compression can introduce decoding dependencies and irregular workloads that limit GPU parallelism. We develop a GPU-native compressed data-processing engine with mixed dispatch, enabling efficient parallel decoding while keeping selected data compressed during H2D transfer. Third, training randomization and downstream processing can offset the benefits of early selection. We integrate locality-aware asynchronous scheduling and direct tensor construction into a PyTorch-compatible pipeline, preserving selection benefits from storage to model input. Experiments on RTX 4090 and H100 GPUs show that FastLens achieves average training speedups of 3.80–4.20× and inference speedups of 6.17–7.95× over the PyTorch DCT baseline.

1 Introduction

Efficient image AI pipelines are essential for large-scale data analytics [28, 38, 53]. Conventional pipelines often decode images on CPUs and transfer expanded data to GPUs, consuming substantial bandwidth and leaving expensive GPUs waiting for inputs. As GPU model computation becomes faster [10, 11, 23, 26, 27, 31, 34, 60], efficient input delivery becomes increasingly important to end-to-end performance [30, 36, 61]. In our experiments, input preparation overhead can reduce training throughput by up to 63% compared with a GPU-resident-input reference, highlighting the cost of input preparation (see § 6.2.4). FastLanes is a high-performance columnar compression and storage format [2, 3]. Therefore, we aim to develop an image AI input system based on FastLanes that co-designs compressed image storage, CPU-GPU transmission, and GPU-side processing to support storage-level selection, compressed transfer, and efficient GPU execution, bridging the gap between data delivery and GPU computation in large-scale image AI.

We highlight three key benefits of designing a selection-aware, GPU-oriented image AI input system, as illustrated in Figure 1 (b). ① *Early selection reduces unnecessary work.* Models may require only cropped regions or selected DCT frequency coefficients, yet conventional pipelines often read and decode more data than needed before applying such selection. We call this problem *late selection*. As shown in Figure 1 (a), this wastes storage bandwidth and CPU resources on data that is eventually discarded. ② *Compressed transfer reduces data movement.* This is particularly important given the large bandwidth gap between GPU memory (3.35 TB/s on H100 SXM) and H2D links (up to 64 GB/s per direction over PCIe 5.0

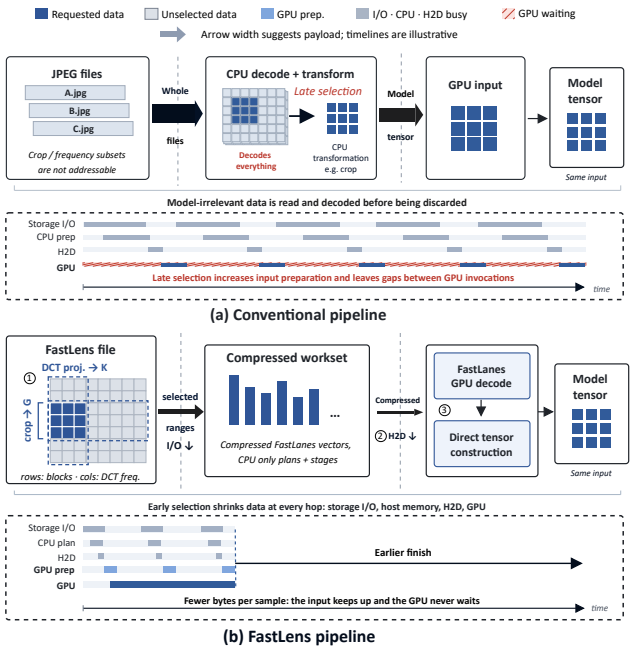


Figure 1: Late selection can leave expensive GPUs waiting for inputs. (a) Conventional pipelines process and transfer data that the model does not need, potentially delaying subsequent batches and extending workload completion time. (b) FastLens pushes spatial and frequency selection into compressed storage, preserves compression through H2D transfer, performs GPU-native decoding, and directly constructs model-ready tensors. Reducing input stalls allows more productive use of GPU capacity and faster completion of the same logical workload. Timelines are illustrative.

x16) [7, 39]. In our evaluation, compressed transfer with FastLanes reduces H2D traffic by 87.7%, from 10.009 GB to 1.232 GB, compared with raw int16 transfer for the same workload (see § 6.4). ③ *GPU-native processing reduces input preparation effort.* GPU parallel decoding reduces CPU work, while direct tensor construction avoids unnecessary intermediate data expansion. Together, these capabilities eliminate data processing and movement throughout the pipeline, helping input delivery keep pace with GPU throughput.

However, building such a GPU-oriented compressed image format and pipeline faces three key challenges. First, *supporting selectivity without sacrificing data locality.* Image models may require only subsets of the stored data, but fine-grained access can lead to scattered reads and poor batch efficiency. A suitable data layout must expose model-driven selectivity while preserving locality for efficient batched processing. Second, *balancing compression efficiency and GPU parallelism.* Keeping data compressed reduces transfer costs, but compression can introduce decoding dependencies and irregular workloads. Efficient GPU processing requires highly parallel decompression while controlling memory consumption and execution overhead. Third, *preserving selection benefits throughout the pipeline.* Model-driven randomness and transformations

can conflict with efficient physical execution, while downstream processing may reintroduce unnecessary computation and data movement. The system must preserve early selection benefits from storage to model input without changing model-visible semantics.

Existing approaches do not jointly address these challenges. JPEG [56] provides compact storage, but its entropy coding limits its fine-grained selective access and parallel decoding. NVIDIA DALI [40, 42] accelerates JPEG decoding and preprocessing on GPUs and supports region-of-interest (ROI) decoding, but still processes data beyond model requirements due to limitations of the underlying format. Frequency-domain vision studies [18, 21, 45, 58] reduce decoding overhead by exploiting DCT coefficients, but do not optimize selective storage access and data delivery together. Image Calculator [52] and Frequency-Store [54] introduce workload-aware image formats and frequency-selective storage, but do not combine selective batched access with GPU-native decompression and end-to-end processing.

To address these challenges, we present ***FastLens***, a GPU-oriented image input system that co-designs a selection-aware compressed image format and a GPU-native processing pipeline. FastLens introduces three key designs. First, we co-design data layout and projection planning to support model-driven selective access while maintaining locality across image batches. Second, we process selected compressed data directly on GPUs through a unified decoding engine, avoiding unnecessary H2D traffic and repeated kernel launches. Third, we decouple model-visible sample order from physical execution and directly construct model-ready tensors, preserving early selection benefits without changing model semantics.

We evaluate FastLens on RTX 4090 and H100 GPUs across multiple image AI workloads. Compared with the PyTorch DCT baseline [46], FastLens achieves average speedups of 3.80–4.20× in training and 6.17–7.95× in inference. It also achieves 99.1% of its GPU-resident training throughput reference on average.

To the best of our knowledge, FastLens is the first image AI input system to unify storage-level spatial-frequency selection, compressed H2D transfer, GPU-native decoding, and direct tensor construction. In summary, we make the following contributions:

- **A batch-friendly compressed image format.** We introduce a cross-image block-major layout with dependency-aware projection pushdown, enabling joint spatial-frequency selection and efficient batched access. (§ 4.1)
- **A GPU-native compressed data-processing engine.** We develop a mixed-dispatch FastLanes decoder that efficiently handles heterogeneous encodings and their dependencies while keeping selected data compressed during H2D transfer. (§ 4.2)
- **An end-to-end selection-preserving image AI pipeline.** We carry model-driven selection through every stage of input preparation, construct model-ready tensors directly from the selected coefficients, and expose them through a PyTorch-compatible pipeline. (§ 4.3 and § 5)

2 Background and Related Work

FastLanes. FastLanes [3] is a new big data file format designed to exploit new hardware capabilities, such as SIMD and SIMT, for maximum decompression throughput. Rather than general-purpose compression methods [12, 20] whose decoding is not data-parallel, FastLanes relies on lightweight encodings [1, 19, 57, 62] over interleaved bit-packed values. FastLanes encodes and decodes data in units of 1024 values at a time called *vectors*. A file is divided into *rowgroups*, and within each rowgroup every column is encoded by an *expression*, a cascade of lightweight encodings. The footer contains metadata for each rowgroup and column, including its expression and the location of its encoded data. Therefore, *vectors* can be located and decoded independently.

This fine-grained access and parallel-friendly execution create an opportunity for image pipelines, where models often require only selected spatial regions or frequency coefficients while benefiting from GPU decoding. However, seizing this opportunity requires mapping model-driven requests to compressed vectors while respecting image semantics and maintaining data locality and batch efficiency, needed for image AI workloads. FastLens addresses this through a selection-aware image layout and dependency-aware projection planning, enabling the required compressed workset to be selectively read, transferred, and decoded on the GPU.

GPU decompression. GPUs offer massive SIMT parallelism but favor regular computation and memory access patterns. Decompression, however, is challenging due to data dependencies that limit parallelism and heterogeneous encodings that require different decoding paths. Prior work has explored GPU decompression [17, 51] and optimized parallel decoding for lightweight encodings [4, 25]. However, these approaches primarily target individual codecs or predefined combinations, whereas model-driven selection produces fine-grained worksets with diverse encoding expressions and decoding dependencies. FastLens addresses this gap through a unified mixed-dispatch GPU decoder that efficiently processes such heterogeneous worksets.

Image input pipelines. Prior systems reduce image-input overhead through faster decoding, caching, and preprocessing. DALI [42] supports CPU- and GPU-based decoding and transformations, while CoorDL [36] coordinates caching and loading to reduce data stalls. Other systems optimize preprocessing graphs and execution schedules [28, 61], and FFCV [32] combines specialized storage with optimized loading and compiled transformations. ML-oriented formats [22, 33, 44] further improve streaming and random access to samples or tensors. While these systems improve data retrieval and preparation, FastLens reduces data that must be read and processed by pushing model-driven selection into compressed storage.

Workload-aware image representations and DCT models. Prior work explores accelerator-friendly image encodings [8, 9] and DCT-domain models that avoid full RGB reconstruction [14, 16, 18, 21, 45, 47, 58]. JPEG provides compact DCT storage but limits fine-grained access [56], while DALI supports ROI decoding [40] but still retrieves complete JPEG files through its standard reader [41]. PCR organizes progressive JPEG scans for runtime fidelity selection, but restricts access to scan prefixes [29]. Image Calculator searches for workload-specific representations [52], whereas Frequency-Store organizes frequency-domain data for different frequency budgets [54]. In contrast, FastLens stores quantized DCT coefficients in a reusable, indexed compressed representation, enabling storage-layer joint spatial–frequency selection at runtime without model-specific storage materialization.

3 Conceptual Framework

In this section, we introduce a conceptual framework to guide the design of FastLens. We first formalize model-driven selection as a logical input request, then characterize how compression granularity and decoding dependencies determine the corresponding physical compressed workset. By making the gap between logical demand and physical work explicit, this framework motivates our storage layout, projection planning, and GPU execution pipeline.

3.1 Logical Input Request Definition

The discrete cosine transform (DCT) [5] is a widely used transform in image compression [48]. JPEG [56] is a canonical example: it partitions each image component into a grid of 8×8 spatial blocks, transforms each block into 64 DCT coefficients, quantizes the coefficients, and entropy-codes the resulting values for compact storage, as shown in Figure 2. We use $b = (i, c, p)$ to denote a source block, where i identifies the source image, c the image component, and $p = (x, y)$ the component-block position. We denote its quantized coefficient at frequency index k by $q_{b,k}$, where $k \in \{0, \ldots, 63\}$ follows JPEG zig-zag order. Thus, JPEG-derived DCT representation naturally consists of two dimensions: *spatial blocks* and *frequency coefficients*. Conventional JPEG decoding reconstructs pixels from these coefficients. DCT-domain vision models, however, can consume tensor representations derived directly from the DCT coefficients, avoiding full RGB reconstruction [14, 16, 18, 21, 45, 47, 58]. Consequently, the blocks and frequency coefficients required by the model can be reasoned about directly, without first reconstructing the complete pixel representation.

Selective model request. DCT-domain model inputs expose two independent dimensions of model-driven selectivity: *spatial selectivity* and *frequency selectivity*, as illustrated in Figure 2 (a). First, *spatial transformations* determine which source blocks can contribute to the model input. For example, a crop uses only the blocks overlapping the selected image region, rather than the complete source image. More generally, each spatial transformation induces a source support over the block grid. We denote the complete set of source blocks required to produce the requested spatial input by G. Second, the *model and input profile* determine which source frequencies are required to construct the requested representation. The DCT concentrates much of the signal energy of natural images into low-frequency components [48], while coefficients at different frequencies encode image variation ranging from coarse structure to finer detail. Prior frequency-domain vision models have shown that selected subsets of DCT channels can be sufficient for downstream tasks [18, 58]. We therefore denote by $K \subseteq \{0, \ldots, 63\}$ the set of source-frequency indices required to construct the requested model input under a particular model or input profile.

Blocks outside G do not contribute to the requested spatial input, and source frequencies outside K are not required to construct the requested representation. Ideally, such excluded data should not propagate through the physical input path. Together, G and K define the logical source subset that the input system must produce.

The logical input request. Both Spatial and frequency selectivity give a relational view of DCT input preparation. Each source block forms a logical row, while frequency indices form columns. For a given model input request, the logical input request is

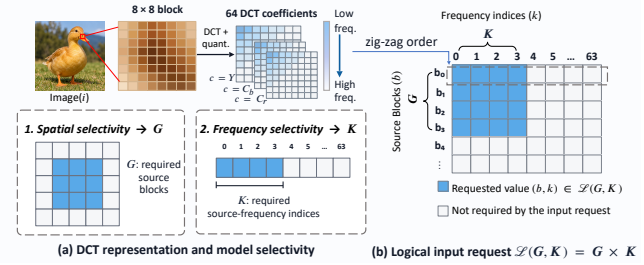


Figure 2: Model selectivity and the logical DCT input request. (a) JPEG-derived DCT data expose two independent selectivity dimensions. Spatial transformations determine the required source-block set G; the model and input profile determine the required source-frequency index set K. (b) Each source block $b \in G$ and frequency index $k \in K$ identifies a requested coefficient $q_{b,k}$; together they form the logical input request $\mathcal{L}(G, K) = G \times K$.

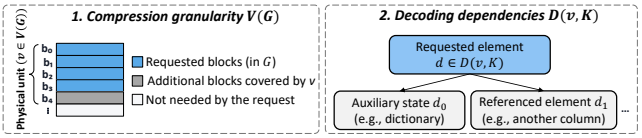


Figure 3: From the logical input request to physical compressed work. Compression granularity expands the requested block set G to the physical encoded units $V(G)$; decoding dependencies introduce extra encoded elements $D(v, K)$.

$$\mathcal{L}(G, K) = \{(b, k) \mid b \in G,\ k \in K\} = G \times K. \tag{1}$$

Figure 2 (b) illustrates this abstraction. The two dimensions vary independently: augmentation can change G across images and epochs, while models or input profiles may require different K.

3.2 Physical Compressed Workset Modeling

The logical projection $\mathcal{L}(G, K) = G \times K$ specifies the logical coefficient values required to construct the requested model input. A compressed representation, however, does not generally expose every (b, k) pair as an independently accessible encoded object. The gap between logical demand and physical work arises from two properties of the representation: *compression granularity* and *decoding dependencies*, as illustrated in Figure 3. First, *compression granularity* determines the smallest physical unit that can be accessed or decoded independently. We use v to denote such a physical encoded unit. Since one unit may cover multiple source blocks, accessing the requested set G may also bring in data from unrequested blocks. Let $V(G)$ denote the set of physical coding units required to cover the requested block set G. Second, *decoding dependencies* can require encoded state beyond the requested values. For a physical unit v, let $D(v, K)$ denote the dependency-closed set of encoded elements required to recover the requested frequencies K. Depending on the compression scheme, these elements may include metadata, dictionaries, exception data, or references to other encoded values. The physical compressed workset is therefore

$$\mathcal{W}(G, K) = \{(v, d) \mid v \in V(G),\ d \in D(v, K)\}. \tag{2}$$

Each pair (v, d) identifies a physical coding unit together with an encoded element required to decode the requested values. Thus,

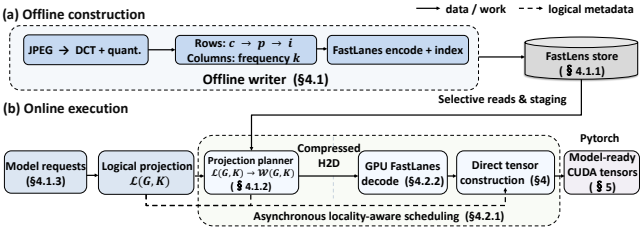


Figure 4: FastLens system overview. By coupling offline storage organization with locality-aware online execution, FastLens preserves model-driven selection throughout the input pipeline while decoupling physical execution order from model-visible sample order.

$\mathcal{L}(G, K)$ describes what the input request logically requires, whereas $\mathcal{W}(G, K)$ describes what the compressed representation requires the system to physically access and decode. In general, $\mathcal{W}(G, K)$ may include state outside the logical source projection: compression granularity can introduce additional logical values, while decoding dependencies can introduce additional encoded state.

***Design implications**. This framework reveals three requirements for efficient selective image input. **First**, the physical layout must support fine-grained selection while preserving data locality and batch efficiency. **Second**, the dependency-closed workset must be efficiently transferred and decoded despite heterogeneous encodings. **Third**, downstream processing must preserve selection benefits without reconstructing unnecessary intermediate data, while maintaining model-visible semantics. These requirements guide FastLens's selection-aware layout and projection planning (§ 4.1), GPU-native compressed execution (§ 4.2), and direct tensor construction (§ 4.3).*

4 FastLens Design

FastLens integrates offline storage construction and online request execution, as illustrated in Figure 4. **(a) Offline construction.** FastLens converts the JPEG dataset into FastLanes-encoded [2, 3] DCT coefficients and builds indexes for selective access. It organizes the data independently of model-specific transformations, allowing the same compressed representation to serve different model requests. **(b) Online execution.** For each model request, FastLens identifies the required compressed workset and transfers it to the GPU in compressed form. It then performs GPU-native decoding and directly constructs model-ready tensors from this data.

4.1 Projection-Aware Layout/Request Codesign

FastLens makes the logical projection $\mathcal{L}(G, K)$ (see § 3) physically addressable through a cross-image block-major layout and dependency-aware projection planning. It organizes DCT blocks as rows and frequency indices as columns, then uses a compact geometry index and encoding metadata to identify the compressed vectors and decoding dependencies required by each request. Together, these mechanisms enable selective reads from compressed storage and locality-aware batched access, as illustrated in Figure 5.

4.1.1 *Cross-Image Block-Major Layout.*

Partition organization. The dataset is divided into partitions, each stored as a FastLanes-format [3] coefficient file, with image metadata and a compact geometry index kept beside it. A dataset-level manifest maps contiguous ranges of stored image indices to partitions of N_p images each ($N_p = 1,024$ in our experiments), which bounds the scope of cross-image organization and lets partitions be built and read independently.

Rows, columns, and vectors. Within a partition, each existing JPEG DCT block $b = (i, c, p)$ forms one logical row, where i is the stored image, c is the component, and p is the component-block position. Rows are ordered by component position on the Morton space-filling curve (i.e., Z-ordered [37, 43]), then by stored image: $c \rightarrow p \rightarrow i$. Blocks with the same (c, p) therefore form a contiguous *position group*. Only existing blocks are stored; a smaller image does not introduce logical padding rows for spatial positions outside its component grid. Each of the 64 DCT frequency indices forms a logical column Q_k, and all columns share the same row ordering. The writer partitions the row stream into *rowgroups*, appending complete position groups without splitting them merely to meet a fixed row count. Rowgroup sizes may therefore vary. Within each rowgroup, each frequency column is encoded as a sequence of FastLanes vectors with capacity $N_{\text{vec}} = 1024$ values. Vectors are formed within a rowgroup and may span position-group boundaries. All columns share the same logical row and vector boundaries, although their encoding expressions and compressed sizes may differ. A partially filled final vector is padded for encoding only; the rowgroup metadata records its valid rows. With $N_p = 1024$ and equal-size images, every position group has exactly 1024 rows: each vector holds one block position (c, p) of all images in the partition, and each rowgroup holds 128 position groups.

4.1.2 *Dependency-Aware Projection Pushdown.*

This section shows how various indexes are used to push projections down to the storage layer.

Image metadata. Each partition stores image metadata separately from the compressed coefficient file. The metadata records the image and component geometry needed to interpret the coefficient rows, including image dimensions, component block dimensions, component identities, and references to shared encoding profiles. Quantization tables are deduplicated within a partition and referenced by ID, so no image-specific geometry or quantization state is embedded in the compressed rowgroups.

Geometry index. Because only existing blocks are stored, the row of block (i, c, p) cannot be derived from a fixed stride: it depends on which images contain the positions stored before it. Let $P(c, p) = \{i \mid \text{image } i \text{ contains block } (c, p)\}$. As component grids are anchored at the origin, image i contains position $p = (x, y)$ iff $x < w_i^c$ and $y < h_i^c$, its block-grid width and height. The distinct widths and heights of a partition thus cut the grid into rectangular *membership cells* whose positions share the same $P(c, p)$. E.g., images of 4×4, 4×2, and 2×4 blocks yield four cells, with $P = \{0, 1, 2\}$ for $x, y < 2$, $\{0, 1\}$ for $x \geq 2, y < 2$, $\{0, 2\}$ for $x < 2, y \geq 2$, and $\{0\}$ otherwise. Per component, the index stores grid extents and these thresholds; per cell, its population $|P|$ and members, encoded by density as an all-present marker, a compressed index list, or a bitmap with rank checkpoints. To locate block (i, c, p), the reader (1) finds the cell of (c, p) and checks $i \in P(c, p)$; (2) obtains start(c, p) from the cumulative row count stored at the

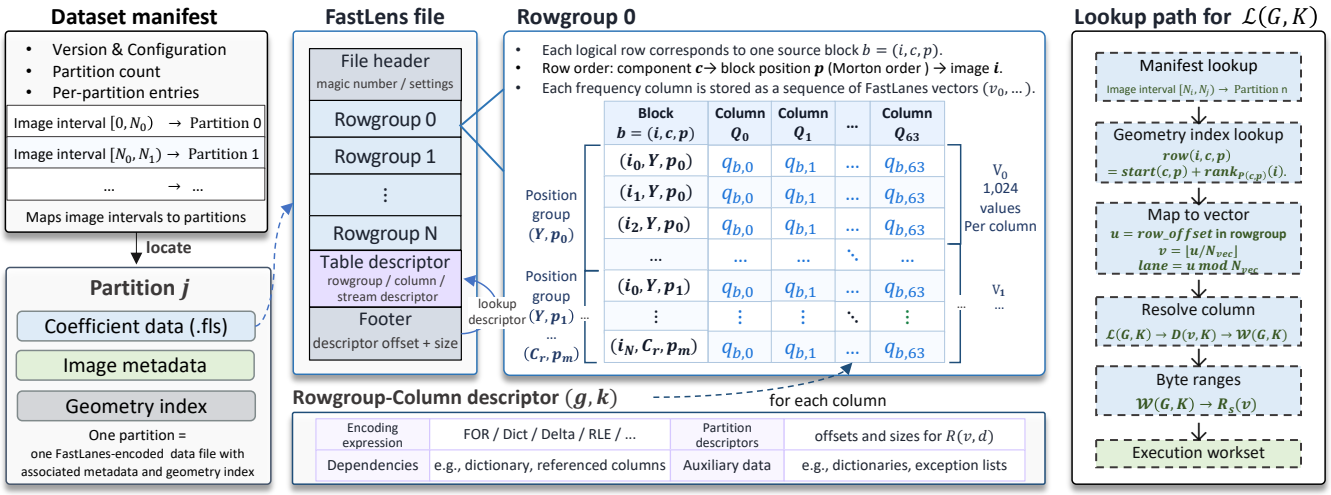


Figure 5: FastLens physical layout and compressed-workset lookup. The dataset manifest maps image-index intervals to partitions, each containing a compressed coefficient file, image metadata, and a geometry/access index (left). Within a partition, existing DCT blocks $b = (i, c, p)$ form logical rows ordered by component c, Morton-ordered spatial position p, and partition-local image i, while the 64 DCT frequencies form columns Q_k (center). Rowgroups contain complete position groups and may have variable row counts. Within each rowgroup, each frequency column is encoded as FastLanes vectors of up to $N_{\text{vec}} = 1024$ logical rows, with encoding-only padding for a partially filled final vector. For a logical request $\mathcal{L}(G, K)$, the geometry index identifies rowgroup–vector–lane locations, while rowgroup–column descriptors enable dependency resolution and compressed byte-range lookup for the physical workset $\mathcal{W}(G, K)$ (right).

nearest preceding *topology checkpoint* (one per rowgroup) on the component's Morton sequence plus the cell populations of the (at most 127) positions in between; and (3) adds the exclusive rank $\text{rank}_{P(c,p)}(i) = |\{j < i \mid j \in P(c, p)\}|$.

$$\text{row}(i, c, p) = \text{start}(c, p) + \text{rank}_{P(c,p)}(i). \tag{3}$$

Since the writer closes rowgroups only between position groups, at a fixed target of 131,072 rows, replaying this rule during step (2) yields the rowgroup and the row's rowgroup-local offset u, giving vector $v = \lfloor u/N_{\text{vec}} \rfloor$ and offset $r_{\text{vec}} = u \bmod N_{\text{vec}}$. If all images of a partition have equal size, as in our ImageNet store (resized to 512×512), there is a single cell and Eq. (3) reduces to $\text{start}(c, p) + i$.

Mapping a Projection to Compressed Ranges The right side of Figure 5 shows how the planner turns a logical request $\mathcal{L}(G, K)$ into the byte ranges of its dependency-closed physical workset $\mathcal{W}(G, K)$ (Equation 2) in four steps, computing $V(G)$, $D(v, K)$, and the byte ranges of $\mathcal{W}(G, K)$.

① **Locate the spatial vectors.** For each requested image, the manifest identifies its partition and partition-local index. The geometry index then maps each required (i, c, p) to its rowgroup, vector v, and within-vector row offset r_{vec} as described above. Requests that fall in the same vector are represented once in the physical vector set $V(G)$, while retaining their individual output destinations.

② **Resolve frequency expressions and dependencies.** For each selected rowgroup and frequency $k \in K$, the planner looks up the corresponding column descriptor. The descriptor identifies the expression and the encoded streams required to reconstruct it. Some expressions reference state beyond the requested column: an equality reference needs the referenced column's streams, and dictionary or patched encodings need dictionary or exception state. Recursively resolving these references for the selected vectors yields $D(v, K)$; such inputs are reconstruction dependencies and do not enlarge K. On the ImageNet training set, 34.641% of vectors use frequent-value coding (one dominant value, typically zero, plus exceptions), 32.537% use Cross-RLE (run-length encoding of repeated values), and 29.224% use patched FOR with bit-packing. Dictionary encodings account for 0.287%, and the remaining 3.311% use constant, uncompressed, equality-reference, or unpatched FOR encodings. Cross-column references occur in 0.075% of all vectors.

③ **Translate selected vectors to byte ranges.** Each encoded stream stores a data region together with cumulative vector endpoints. Let o_{rg} be the rowgroup file offset, o_s the stream's offset within the rowgroup, and $e_s[v]$ the cumulative end offset of vector v in stream s. Then the bytes for that vector are $R_s(v) = \left[o_{\text{rg}} + o_s + e_s[v-1],\ o_{\text{rg}} + o_s + e_s[v]\right)$, with $e_s[-1] = 0$. Shared streams, such as expression state shared across vectors, are referenced as shared ranges rather than replicated for each selected vector. Since compressed vectors have variable length, these endpoints are what make vectors addressable; a run of consecutive selected vectors maps to one range given by its two endpoints.

④ **Form the execution workset.** The planner sorts the ranges of the selected vectors and their dependency streams and coalesces overlapping or adjacent intervals; shared state that is already loaded is not re-issued. The coalesced ranges are read and packed contiguously into a staging buffer (*compact staging*), so H2D carries only selected bytes rather than ranges at their source-file offsets. The planner also emits (i) one decode work item per selected vector–column (§ 4.2.2), (ii) a *vector remap* giving each selected vector's

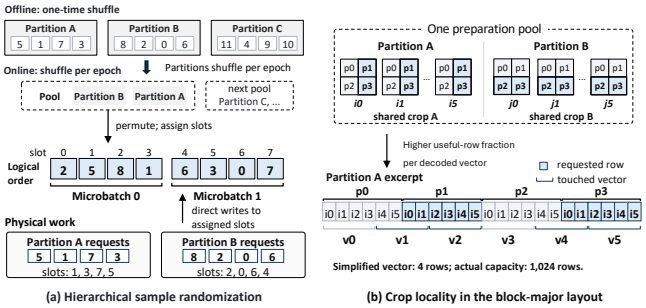


Figure 6: Locality-preserving training randomization and augmentation. (a) One-time dataset shuffling and epoch-wise partition and within-pool permutations enable locality-aware execution while preserving model-visible sample order. (b) Epoch-specific, partition-shared crops align spatial requests across images, improving useful-row utilization of block-major FastLanes vectors.

position in compact decoded storage, and (iii) per-sample *source bindings*: logical slot, image, crop origin, and flip. No per-coefficient mapping is materialized: construction kernels derive source and destination addresses on demand from these bindings and the model profile (§ 4.3). The resulting workset therefore binds the logical projection $\mathcal{L}(G, K)$ to the compressed ranges and decoding state required for execution. Importantly, FastLens does not require every coefficient or image block to be an independently compressed object: the layout exposes the actual vector granularity and encoded-stream dependencies, and the planner realizes the request at that physical granularity.

4.1.3 Locality-Preserving Training Requests. Training requires randomized sample order and augmentation, while the block-major layout benefits from storage and spatial locality. Arbitrary physical accesses would destroy the locality the layout exposes. We reconcile these requirements by decoupling the *model-visible training schedule* from *locality-oriented physical execution.*

Hierarchical Locality-Preserving Randomization. Figure 6 (a) shows the two levels of randomness. *Offline*, we shuffle the dataset once before cutting it into fixed partitions [59]. *At each epoch*, we (1) shuffle the partition order, (2) group every m consecutive partitions ($m = 4$) into a *preparation pool*, the unit that FastLens plans and decodes together, and (3) permute the samples within each pool. Step (3) assigns each sample a logical slot; consecutive slots form the model's microbatches. The planner then executes a pool partition by partition, across microbatch boundaries, while tensor construction writes each result to its slot (§ 4.2.1). Relative to a full shuffle, the only restriction is that two samples can share a microbatch only if their partitions share a pool in that epoch; this pairing changes every epoch, and stored data is never reorganized. § 6.2.5 confirms that convergence is maintained for our workloads.

Locality-Preserving Spatial Augmentation. Independently sampled crops would touch unrelated block positions. Our default policy therefore samples one block-aligned crop box per partition and epoch and applies it to all of the partition's images (Figure 6 (b)). Because the block-major layout stores each (c, p) of all images contiguously, every vector touched by a shared crop is fully useful, since all images of the partition request that position. Only the crop is shared: horizontal flip is drawn per sample and applied during tensor construction (§ 4.3), and RandAugment per sample in coefficient-domain postprocessing. A pool spanning several partitions thus contains several crops.

4.2 Compression-Preserving Workset Execution

The storage layer reduces a model request to a compressed workset. Our goal on the GPU is to preserve this reduction until the requested model representation is produced. This requires solving three problems. First, the block-major layout pays off only when many images are processed, yet the model consumes small batches in a randomized order (§ 4.2.1). Second, a workset contains many small vectors with heterogeneous FastLanes expressions, which must be decoded without repeated host dispatch (§ 4.2.2). Third, decoded coefficients must become model tensors without reconstructing the dense grids that projection eliminated (§ 4.3).

4.2.1 Asynchronous Workset Scheduling and Transfer. A model invocation consumes a microbatch of, e.g., 64 samples, whereas block-major vectors are used efficiently only when a request touches many images per position. We therefore prepare a whole *preparation pool* of four partitions (4,096 images; § 4.1.3) at once and serve the *model microbatches* as views into its output.

Decoupling physical and logical order. Each sample's logical slot and its crop, flip, and augmentation decisions are fixed before planning. The planner may then reorder reads and decode work freely, and construction kernels write each sample directly to its slot in a pool-sized output tensor; microbatch j is the view on slots $[jB, (j{+}1)B)$. An image occurring twice thus shares physical source data while keeping distinct augmentations.

Algorithm A Per-Epoch Pool Scheduling and Workset Preparation

```
1  procedure RunEpoch(seed, epoch)
2    P ← Permute(allPartitions, seed, epoch)
3    parallel: Producer(P, seed, epoch) ‖ Consumer()
4  procedure Producer(P, seed, epoch)
5    for all (pool_id, members) ∈ Enumerate(GroupsOfFour(P)) do
6      order ← Permute(ImagesIn(members), seed, epoch, pool_id)
7      requests ← [ ]
8      for all (slot, image_id) ∈ Enumerate(order) do
9        crop ← Crop(seed, epoch, PartitionOf(image_id))
10       flip ← Flip(seed, epoch, image_id)
11       Append(requests, (image_id, slot, crop, flip))
12     SubmitPrepare(requests, maxLive = 2)
13 procedure Consumer
14   for all pool ∈ PreparedPoolsInOrder() do
15     for b ← 0 to pool.size − 1 step 64 do
16       ConsumerStreamWait(pool.ready_event[b/64])
17       YieldView(pool.tensor[b : min(b + 64, pool.size)])
18     ReleaseAfterConsumersComplete(pool)
19 procedure Prepare(requests)
20   plan ← PlanByPartitionAndRowgroup(requests)
21   for all workset ∈ BoundedWorksets(plan, 64 rowgroups, 512 MiB)
     do
22     ReadSelectedRangesAndStage(workset)
23     UploadCompressedRangesAndDescriptors(workset)
24     decode_slots ← PackBySemanticLaneWidth(workset.work_items)
25     DecodeMixed(decode_slots)
26     ConstructRequestedOutputs(workset, requests.logical_slots)
27   PostprocessMicrobatchesAndRecordReadiness()
```

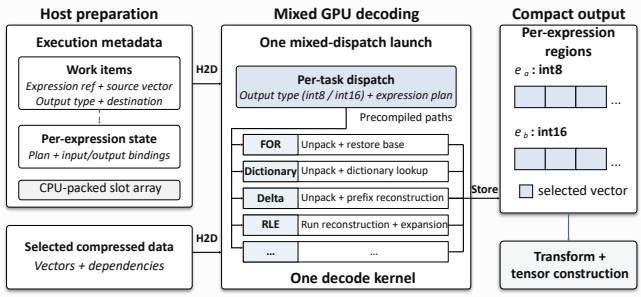


Figure 7: GPU-native FastLanes decoding with mixed dispatch. The CPU prepares work items, per-expression state, and packed slots, which accompany the selected compressed workset across H2D (left). Within one decode-kernel launch, per-task dispatch uses the output type and expression plan to select a precompiled reconstruction path. Heterogeneous expressions and output types share the launch (center). The decoder writes `int8`/`int16` coefficients compactly within per-expression output regions, which feed transformation and tensor-construction kernels (right).

Algorithm A summarizes this flow. The producer fixes each sample's pool slot and augmentation parameters (lines 4–12); PREPARE then executes the pool as bounded worksets (≤64 rowgroups or 512 MiB) in partition/rowgroup order, writing each output to its slot (lines 19–27); and the consumer yields slot-ordered microbatch views as they become ready (lines 13–18). Physical execution order is thus decoupled from model-visible order without a final shuffle.

Overlapping preparation and model execution. A pool is executed as a sequence of bounded worksets. Host threads plan and read upcoming worksets while the GPU processes the current one through H2D transfer, FastLanes decoding, and tensor construction on four CUDA streams ordered by events. Training postprocessing, when enabled, runs on a fifth stream shared across pools. The model's stream (consumer) waits only on the readiness event of the microbatch it consumes, and at most two pools are in flight, so one pool is consumed while the next is prepared.

Bounding temporary state. Only the final projected or Y/CbCr outputs are pool-sized, since microbatches are views into them (4.93 GB for a 4,096-sample MV2-DCT24 pool). Compressed device buffers and decoder scratch scale with the submitted workset, and augmentation workspace with the microbatch. Similar to PyTorch's `record_stream`, a pool's storage is reclaimed only after all application tensors referencing it are released and all registered consumer streams have finished, so consumption is asynchronous without global device synchronization.

4.2.2 GPU-Native FastLanes Decoding. The first GPU stage reconstructs the selected quantized DCT coefficients. The challenge is that vectors selected by one request may use different FastLanes expressions, encoded states, and output types. Issuing a separate host launch for each expression would turn fine-grained selection into many small GPU invocations. We instead prepare a common task representation on the CPU and execute heterogeneous tasks through a mixed-dispatch kernel. Using the dependencies resolved during projection planning, we create work items that identify their source expression and vector range, output type, and compact destination. Each work item references shared per-expression state holding the reconstruction plan and the input/output bindings (Figure 7, left). We transfer this execution metadata with the selected vectors and their encoded dependencies, keeping the coefficient data compressed through H2D.

Mixed on-GPU dispatch. Within the decode kernel, we dispatch first on a work item's output type and then on its expression plan to select a precompiled reconstruction path (Figure 7, center). This per-task selection allows heterogeneous expressions and `int8`/`int16` outputs to share a launch while retaining expression-specific reconstruction. The output type also determines a work item's execution width. FastLanes interleaves a vector's values into independent *lanes* that can be decoded in parallel [2]: an `int8` vector forms 128 lanes and an `int16` vector is 64 lanes, with each lane mapping naturally to one GPU thread. We therefore pack work items into *slots* on the CPU before launch. Each slot is assigned to a CUDA thread block with 128 threads and contains either one 128-lane `int8` work item or up to two compatible 64-lane `int16` work items, so no thread idles. In the latter case, each half spans two full warps, so the two tasks execute independently without intra-warp divergence. Within a slot, the work item identifies its value type and FastLanes plan, and we dispatch to the corresponding reconstruction path.

Composable reconstruction. Rather than writing one decoder per expression, we compose reusable operations: *unpacking*, *base or dictionary restoration*, *exception patching*, and *run expansion*. Each supported expression plan is instantiated as a specialized reconstruction path at compile time, so codec intermediates stay in registers where possible. The decode stage writes only selected vectors to compact `int8` or `int16` storage in global memory: vector w occupies the 1024 values at offset $1024 \cdot w.\text{outvec}$ of its expression's output region (Figure 7, right). Coefficient $q_{b,k}$ is thus found at offset $1024 \cdot \text{outvec} + r_{\text{vec}}$ of that region, with outvec from the vector remap. This representation is the boundary between codec reconstruction and model-specific tensor construction.

4.3 Direct Tensor Construction

GPU decoding produces selected quantized DCT coefficients, which must become model-ready tensors with the required geometry, channels, numerical representation, and layout. A conventional approach first materializes dense transformed FP32 grids over all frequencies and then gathers and normalizes the model channels; this simplifies addressing but re-expands exactly the data that selection removed. FastLens instead constructs the tensors directly from the compact decoded coefficients, along one of two paths (Figure 8).

Single-source direct construction. When every output value has exactly one source coefficient, as for crop, flip, and channel selection, the kernel computes for each output element its source block from the sample's binding (crop origin, flip), locates the coefficient via the vector remap (§ 4.2.2), and negates it if the sample is flipped and its horizontal frequency is odd, since mirroring an 8×8 block multiplies horizontal frequency u by $(-1)^u$. It then dequantizes with the image's quantization table, clamps, normalizes, and stores the value at its NCHW position (Figure 8 (a)). Only requested channels are produced.

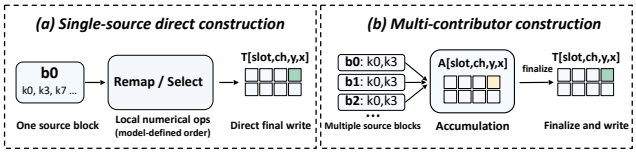


Figure 8: Selection-preserving tensor construction in FastLens. Directly constructible outputs apply local transformations and write directly to assigned tensor locations, while multi-contributor outputs accumulate only over the requested output geometry and channels. Both paths avoid materializing a dense transformed coefficient grid.

Multi-contributor construction. Resize computes each output coefficient as a linear combination of coefficients from several source blocks and frequencies (DCT-domain mixing). FastLens keeps FP32 accumulators only for the requested output geometry and channels, adds contributions as source coefficients are processed, and finalizes (rounding, saturation, affine conversion, normalization) once all contributions are present (Figure 8 (b)). Single-writer mappings, such as integer upscaling where each output has one source block, skip accumulation and finalize directly.

Preserving numerical semantics. Eliminating intermediate materialization does not permit arbitrary reordering of numerical operations. FastLens preserves the required order of dequantization, configured clamping, DCT-domain mixing, and final rounding, saturation, affine conversion, and normalization. We fuse transformation with final tensor construction only where this preserves these semantics. Final FP32 model inputs, including normalized and Mixup-mixed training tensors, agree element-wise with the PyTorch DCT baseline to within $|x - x_{\text{ref}}| \leq 10^{-5} + 10^{-5}|x_{\text{ref}}|$, where x_{ref} is computed with the same rounding, saturation, and normalization semantics. Training uses the same native outputs and optionally applies coefficient-domain augmentation, normalization, Mixup, and target construction in a subsequent microbatch-scoped postprocessing stage.

5 Implementation

PyTorch-compatible integration. FastLens provides an iterable input pipeline that follows the standard batch-at-a-time PyTorch programming pattern. Specifically, the applications construct a `FastLensPipeline`, iterate over prepared batches, and pass the returned CUDA tensors directly to the model. Replacing an existing DCT input path only changes the input initialization and batch fields; the model forward pass, loss computation, backpropagation, and optimizer logic remain standard PyTorch. An example CNN training loop is shown in List 1. Inference uses the same interface with an explicit request schedule instead of a training epoch.

Model-specific input contracts. The common iterator does not imply that every model consumes the same tensor representation. We use a *model profile* to separate model-specific input requirements from the storage and execution engine. A profile resolves the output geometry, required source coefficients, model-visible channels, normalization, and output representation needed by a particular model. For the CNN workloads, the profile produces a projected FP32 tensor in NCHW layout. Transformer profiles instead produce the luminance and chrominance coefficient grids expected by the model. These tensors are exposed as standard PyTorch CUDA tensors, so no FastLens-specific operator is required in the model forward pass.

Listing 1: DataLoader-like use of FastLens for CNN training.

```
with FastLensPipeline(
    manifest, profile=profile, batch_size=64,
    training=policy) as loader:

    for epoch in range(epochs):
        for batch in loader.start_epoch(epoch):
            logits = model(batch.projected)
            loss = loss_fn(logits, batch.targets)
            loss.backward()
            optimizer.step()
```

An important distinction is that the coefficients needed to *construct* an output are not necessarily identical to the channels finally exposed to the model. Under resize, for example, an output channel at frequency k mixes source frequencies other than k, which may be absent from the projected tensor. The profile therefore specifies source frequencies K separately from the model-visible channels. Batches are views over the pool's CUDA storage, not copies; applications consuming them on another stream register it with the batch so the storage stays live (§ 4.2.1).

6 Evaluation

6.1 Experimental Setup

Baselines. We compare FastLens with PyTorch [46], DALI [42], CoorDL [36], FFCV [32], L3 [8], Image Calculator (IC) [52], and Frequency Store (FS) [54]. For FFCV, we test both compressed (FFCV JPEG) and uncompressed (FFCV raw) storage. Since IC and FS are not open source, we adapt their published storage organizations to a common DCT input contract. The source-coefficient stores retain the JPEG's quantized 8×8 DCT coefficients and quantization tables. *IC* compresses each image independently; *IC-batched* compresses multiple images as a contiguous image-major array. *FS* stores cross-image frequency anti-diagonals (D) as separately compressed columns (15 per 8×8 block). Our IC implementation does not include representation search.

Comparison Settings. We use two complementary comparison modes. In *matched DCT* comparisons, we fix the source data, model inputs, model, and numerical semantics within each configuration. Full-frequency comparisons evaluate PyTorch DCT and FastLens, while frequency-selective experiments additionally include IC and FS. Each system retains its own input-processing path under the same CPU, host-memory, and GPU resource budgets, so throughput reflects the complete input pipeline rather than individual storage mechanisms. In *deployment* comparisons, conventional baselines execute RGB models while FastLens executes DCT models; these results compare complete deployment configurations rather than input paths alone.

Measurement methodology. All throughput measurements exclude warm-up and profiler instrumentation. Inference processes the full 50K-image validation set. Training uses 65,536-image windows and reports the median of three trials; Mechanism studies use 16,384-image windows, while input-only layout and selectivity experiments use 4,096 images. Unless otherwise stated, the input

Table 1: Evaluated workloads.

Workload	Params. (M) RGB	Params. (M) DCT	DCT input	Top-1 (%) RGB	Top-1 (%) DCT
ViT-Ti	5.72	5.64	$Y: 28^2,\ C: 14^2$	74.09	75.14
SwinV2-T	28.35	28.34	224 : $Y: 28^2,\ C: 14^2$ 256 : $Y: 32^2,\ C: 16^2$	79.01	79.37
MV2-DCT24	3.50	3.50	24×112^2	70.75	67.70
MV2-DCT32	3.50	3.50	32×112^2	70.75	69.49
RN50-DCT24	25.56	25.53	24×56^2	74.78	75.69
RN50-DCT64	25.56	25.55	64×56^2	74.78	74.15
eFUN-DCT†	5.29	4.23	192×28^2	76.77	73.35

Note. Top-1 is validation classification accuracy. For ViT/Swin, Y and C are the 8×8 DCT-block grids of luminance and chrominance (Cb/Cr). Other DCT inputs show channels×spatial size. RGB inputs are 3×224^2 except Swin's 256 setting (3×256^2). †eFUN's RGB reference is EfficientNet-B0.

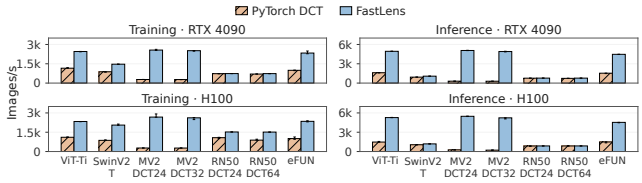


Figure 9: Matched DCT training and inference throughput. Both paths use identical DCT models and logical workloads. MV2 and RN50 denote MobileNetV2 and ResNet-50.

cache and model execution are warmed before timing. Frequency-selective inference uses sustained validation-set replays, while compact-input training follows a separate short-window protocol. We describe these settings alongside their results.

Dataset and workloads. We use ImageNet-1K [13, 49], with 1.28M training and 50K validation images. Following RGB No More [45], we resize images to 512×512 and store them as JPEGs. Table 1 summarizes the evaluated DCT workloads and their RGB deployment references. ViT-Ti [15, 45], SwinV2-T [35, 45], MobileNetV2 [50, 58], and ResNet-50 [24, 58] use architecture-matched RGB counterparts. For eFUN [18], EfficientNet-B0 [55] is a deployment reference.

Platform. We evaluate FastLens on a dual-socket server equipped with two Intel Xeon Gold 5318Y CPUs (48 physical cores) and 2 TiB of host DRAM. The server contains an NVIDIA GeForce RTX 4090 with 24 GiB GDDR6X memory and an NVIDIA H100 with 80 GB HBM3 memory. The system uses CUDA 12.8 and PyTorch 2.11; baseline versions include DALI 2.2 and FFCV 1.0.2.

6.2 End-to-End Performance

6.2.1 Matched DCT Performance. We first compare PyTorch DCT and FastLens under identical full-frequency DCT workloads. Both paths retain all 64 source coefficients per JPEG DCT block.

PyTorch DCT Comparison. Figure 9 compares PyTorch DCT and FastLens across seven workloads. FastLens achieves average speedups of 3.80× on RTX 4090 and 4.20× on H100, reaching 9.85× for MV2-DCT24 on H100. In inference, average speedups are 6.17× and 7.95×, respectively, with a maximum of 24.42× for MV2-DCT32

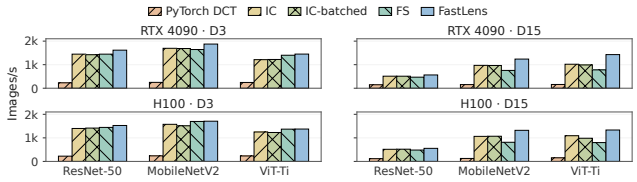


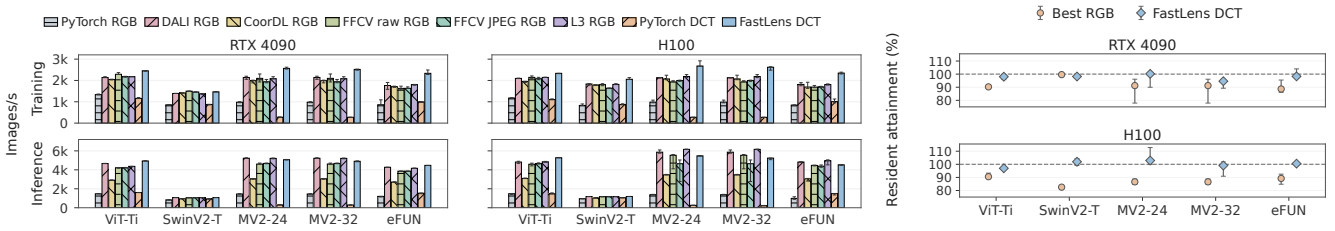
Figure 10: Compact-input training throughput. D3 and D15 use $3 \times 84 \times 84$ and $3 \times 224 \times 224$ inputs. IC uses offline-prepared mosaics; FS and FastLens construct inputs online.

on H100. We make three observations. First, input-intensive workloads, such as MobileNetV2, benefit most because FastLens eliminates costly CPU-side decoding and preprocessing of large DCT inputs. Second, gains are more modest when input preparation is less dominant, as seen in SwinV2-T and ResNet-50 inference (1.01–1.16×). Third, this balance shifts with GPU capability. On H100, training speedups increase from 1.67× to 2.39× for SwinV2-T and from 1.01–1.05× to 1.42–1.70× for ResNet-50, as faster model execution exposes CPU-side input bottlenecks. Together, these results show FastLens improves GPU utilization by reducing input preparation overhead, particularly when it limits end-to-end throughput.

6.2.2 Frequency-Selective DCT Performance. We evaluate FastLens's support for different frequency requirements with two input configurations: shape-preserving selection and compact input. The former keeps model computation fixed to examine selective data loading; the latter evaluates end-to-end training with smaller inputs.

Shape-Preserving Selection. We evaluate three pretrained models with frequency budgets $D \in \{4, 6, 8\}$, retaining 10, 21, and 36 DCT coefficients per block, respectively. Unselected coefficients are zero-filled before preprocessing, preserving the model input shape and checkpoint. PyTorch DCT extracts coefficients from complete JPEGs, while IC prepares separate stores for each budget. FS and FastLens instead serve all budgets from reusable full-frequency stores, with FastLens performing GPU-native processing. All methods produce identical inputs within each budget. We measure sustained FP32 inference throughput over the 50K validation images using batches of 64 and the same CPU and memory budgets. Table 2 reveals three insights. *First*, FastLens efficiently supports multiple frequency budgets using a single compressed representation. Unlike IC, which requires separate precomputed stores, FastLens selects the required coefficients at runtime without materializing model-specific inputs. *Second*, FastLens delivers substantially higher throughput for input-intensive workloads. Across the three budgets, it achieves 6.8–7.5× the throughput of the fastest competing baseline for MV2-DCT24 and 3.4–3.7× for ViT-Ti. These results demonstrate the effectiveness of combining selective compressed access with GPU-native processing. *Third*, the benefits depend on the workload. All RN50-DCT24 paths achieve similar throughput (876–894 images/s), indicating that faster input preparation offers limited gains when model execution dominates.

Compact Input. Following Image Calculator [52], we evaluate compact DCT inputs that reduce the model's input dimensions. Using JPEG-B8 coefficients, $D = 3$ and $D = 15$ yield $3 \times 84 \times 84$ and $3 \times 224 \times 224$ inputs, respectively. Each 8×8 block of the 224-pixel



(a) Training and inference throughput. **(b) Training tput relative to in-GPU execution.**

Figure 11: Deployment throughput and GPU-resident attainment. (a) FastLens leads training in 9 of 10 configurations and averages 97.7% of the fastest RGB inference throughput. (b) Best RGB selects the fastest of six RGB paths per round. The dashed line marks 100% attainment. FastLens achieves an average of 99.1% of its GPU-resident reference.

Table 2: Shape-preserving frequency selection on H100. Inference throughput is measured in images/s (higher is better).

Method	MV2-DCT24			RN50-DCT24			ViT-Ti		
D	4	6	8	4	6	8	4	6	8
PyTorch DCT	272	272	228	888	889	886	1139	1176	1243
IC	307	297	283	891	888	876	1290	1122	982
IC-batched	343	329	321	891	886	885	1389	1200	1041
FS	342	339	330	894	891	889	1506	1357	1211
FastLens	2574	2402	2254	882	881	879	5151	4842	4559

Note. D counts retained low-frequency DCT diagonals per 8×8 block.

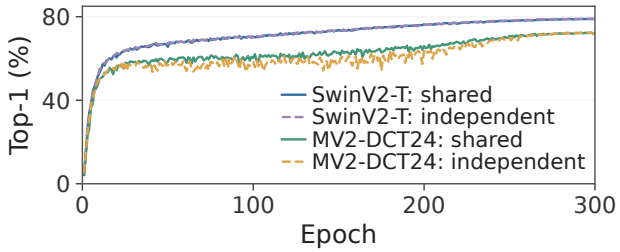


Figure 12: FastLens training convergence: validation Top-1 accuracy over 300 epochs for SwinV2-T DCT & MobileNetV2-DCT24 with the locality-preserving, pool training policy.

crop (28×28 blocks) contributes its top-left $D' \times D'$ coefficient tile ($D' = 3$ for $D = 3$, 8 for $D = 15$), tiled spatially per Y/Cb/Cr channel; FastLens builds this mosaic with the single-source path of § 4.3. IC materializes fixed-geometry inputs offline, whereas FS and FastLens construct them online from reusable stores. We measure actual FP32 training throughput without evaluating convergence. Figure 10 reveals three insights. *First*, FastLens achieves 6.87× higher throughput than PyTorch DCT and 13% higher throughput than the strongest IC/FS baseline on average. Despite performing input construction online, FastLens matches or exceeds specialized offline materialization, demonstrating the effectiveness of its online compressed-data processing pipeline. *Second*, its advantage over IC/FS increases from 5% at $D = 3$ to 21% at $D = 15$. This suggests that GPU-native decoding and direct tensor construction become more valuable as input volume grows, whereas CPU preparation is already competitive for small inputs. *Third*, H100 outperforms RTX 4090 in only one of six paired configurations, suggesting that peak GPU compute is not the sole determinant of throughput.

6.2.3 Deployment Performance. Matched comparisons above isolate the input system. We now ask a different question: *how competitive is the complete DCT deployment relative to conventional RGB pipelines?* Table 1 summarizes their configurations and checkpoint accuracies. ResNet is excluded because its RGB and DCT variants differ in downstream computation, making the deployment throughputs not directly comparable. For training, we compare FastLens against PyTorch, DALI, CoorDL, FFCV, and L3. RGB baselines share the same training settings within each model family, while FastLens follows the corresponding DCT training configuration. Figure 11 (a) shows that FastLens achieves the highest throughput in 9 of 10 configurations, averaging 2.41× that of PyTorch RGB and 19–26% higher throughput than optimized RGB pipelines. The sole exception is SwinV2-T on RTX 4090, where FFCV raw is 2.1% faster but requires 1,075 GB of storage, approximately 10× that of FastLens (107 GB; Table 3). Overall, FastLens achieves competitive or superior DCT training throughput with substantially lower storage costs than raw RGB storage. Training convergence is evaluated separately in § 6.2.5. For inference, FastLens achieves 97.7% of the fastest RGB inference throughput on average, demonstrating that optimized DCT input processing can substantially narrow the performance gap with RGB deployments.

6.2.4 Training Throughput Relative to GPU-Resident Execution. To assess input-pipeline overhead, we compare end-to-end training throughput with GPU-resident execution that repeatedly processes eight preprocessed microbatches (512 samples), excluding input preparation and H2D transfer. Since inputs are reused, this serves as an execution reference rather than a strict upper bound. Figure 11 (b) reports throughput attainment over 65,536 image executions. Across ten configurations, FastLens achieves an average of 99.1% of its GPU-resident reference, compared with 89.6% for the best RGB pipelines. In contrast, PyTorch RGB on H100 SwinV2-T falls 63% below its resident reference. Ratios above 100% reflect input reuse and measurement variability. These results show that FastLens largely removes input-pipeline bottlenecks, bringing end-to-end training throughput close to GPU-resident execution.

6.2.5 Training Convergence. We evaluate convergence under the FastLens training policy (§ 4.1.3), which decouples logical sample order from locality-oriented physical scheduling. Models are trained from scratch for 300 epochs on eight A800 GPUs using BF16, a

Table 3: ImageNet-1K storage footprint. GB includes metadata, indexes, and padding; ×JPEG is the ratio to original JPEG storage for the same split.

		Validation (50K)		Training (1.28M)	
Type	Representation	GB	×JPEG	GB	×JPEG
JPEG	JPEG	2.052	1.00	50.601	1.00
	FFCV (JPEG)	2.180	1.06	51.259	1.01
Coeff.	FS	2.808	1.37	69.789	1.38
	FastLens	4.293	2.09	107.317	2.12
	IC-batched	4.814	2.35	119.621	2.36
	IC-shared	4.858	2.37	120.737	2.39
RGB	L3	29.250	14.26	737.019	14.57
	FFCV (raw)	41.953	20.45	1075.165	21.25

Coeff. stores quantized JPEG DCT coefficients; IC stores full-frequency coefficients.

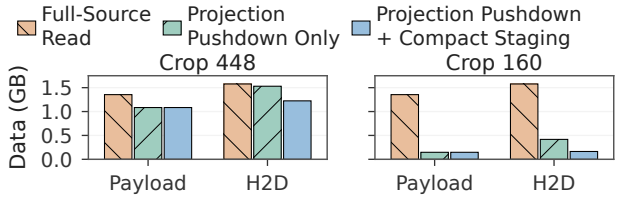


Figure 13: Spatial selection and compact staging on H100. We show issued payload & workset H2D for 448/160 pixel crops.

microbatch size of 64, an effective batch size of 1,024, and the configured crop, RandAugment, and Mixup recipe. Figure 12 reports unsmoothed Top-1 accuracy on the full 50,000-image validation set. Both partition-shared and per-sample independent cropping support convergence across the two architectures. For SwinV2-T DCT, shared and independent crops reach final Top-1 accuracies of 78.888% and 79.052%, respectively, with best checkpoint accuracies of 78.894% and 79.098%. For MobileNetV2-DCT24, shared and independent crops reach final accuracies of 71.976% and 71.764%, with best checkpoint accuracies of 72.368% and 72.440%, respectively. Across both architectures, the results demonstrate that FastLens supports stable convergence over full training schedules while enabling locality-oriented input preparation.

6.3 Data Movement and Resource Costs

Storage. Table 3 highlights the trade-off between storage footprint and input-processing efficiency. JPEG-based pipelines require little storage overhead but retain decoding costs, while RGB-oriented formats such as L3 and FFCV raw trade much larger storage footprints (14.57–21.25× JPEG) for faster data preparation. Coefficient-oriented formats offer a middle ground: FS requires 1.38× JPEG, while our full-frequency IC adaptations require 2.36–2.39×, which may differ from the workload-specific representations chosen by the original IC system. FastLens requires 2.12× JPEG storage, balancing compact storage with selective access and efficient GPU processing. Although FS is more storage-efficient, FastLens delivers higher throughput on input-intensive workloads, showing the importance of jointly optimizing storage, access, and GPU processing. Overall, these results show that FastLens achieves a favorable balance between storage efficiency and end-to-end performance.

Host-to-Device Traffic Figure 14 compares total profiled host-to-device (H2D) traffic per image. FastLens moves 0.057–0.058 MB/image

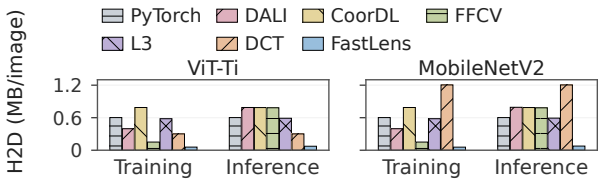


Figure 14: H2D traffic on RTX 4090 (MB/image; lower is better). DCT denotes PyTorch DCT; MobileNetV2 uses DCT24 for the DCT paths. FFCV raw and JPEG transfer identical volumes and share one entry.

in training and 0.074–0.077 MB/image in inference, the lowest volumes among the evaluated paths. Relative to the five RGB deployments, these volumes are resp. 62–93% and 87–91% lower. This shows that FastLens's selective read and compressed H2D can dramatically reduce data transfer, saving precious PCIe bandwidth.

6.4 Ablation Experiments Across the Pipeline

We evaluate whether model-driven input reduction persists across the physical pipeline and improves end-to-end performance. We isolate three boundaries: (1) spatial selection and host staging, (2) compressed H2D transfer and GPU decoding, and (3) final tensor construction. Unless otherwise stated, experiments use MobileNetV2-DCT24 with 16,384 images, FP32, and batch size 64, comparing matched paths that produce identical logical model inputs.

Projection Pushdown and Physical Data Movement. We evaluate how spatial and frequency pushdown reduce physical data movement while preserving the same logical model input. In the deployment training experiment on RTX 4090 (§ 6.2.3), crop pushdown saves 1.253–1.408 GB of physical reads (22.53–24.71%) and 2.246–2.714 GB of H2D traffic (34.96–42.60%) across five workloads. The planned FastLanes vector count also drops by 44.60% (412,507 to 228,541). To isolate these effects, Figure 13 compares three paths on H100 using MobileNetV2-DCT24: *full-source* reads all blocks; *projection pushdown only* reads only crop-relevant blocks but stages them at their source-file offsets; and *pushdown + compact staging* additionally packs the selected ranges contiguously before H2D (§ 4.1.2, step ④). For 448- and 160-pixel crops, spatial selection reduces issued reads from 1.354 GB to 1.084 and 0.147 GB, respectively. Compact staging further reduces H2D traffic from 1.530 to 1.224 GB and from 0.417 to 0.164 GB (H2D also carries decode descriptors, work-item lists, and staging-buffer padding, including unselected gaps in the source-layout path). For frequency selection, early pushdown on RTX 4090 at $|K| = 8$ reduces issued payload from 0.346 to 0.141 GB (59%) and H2D traffic from 0.402 to 0.147 GB compared with GPU-side filtering of all 64 columns. Together, this shows that FastLens translates spatial and frequency selectivity into reduced I/O, while compact staging preserves its savings across H2D. § 6.5.2 examines the end-to-end benefits under reduced storage bandwidth.

Compressed H2D transfer and GPU decoding. We next test whether the selected workset can remain compressed during H2D transfer without making GPU decoding a new bottleneck. Holding the selected coefficients, physical grouping, and model input fixed, we compare raw int16 transfer with FastLens (Figure 15). On both GPUs, FastLens reduces issued reads from 10.527 to 1.084 GB and

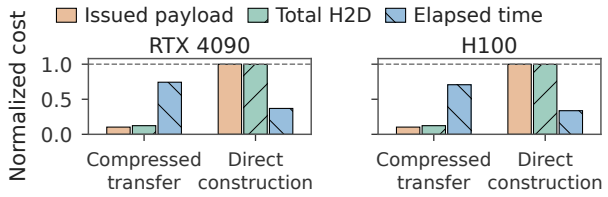


Figure 15: Effects of compressed transfer and direct tensor construction. Each metric is normalized to its matched control; lower is better. Compressed transfer reduces both data movement and elapsed time, whereas direct construction achieves 2.7–3.0× throughput improvement.

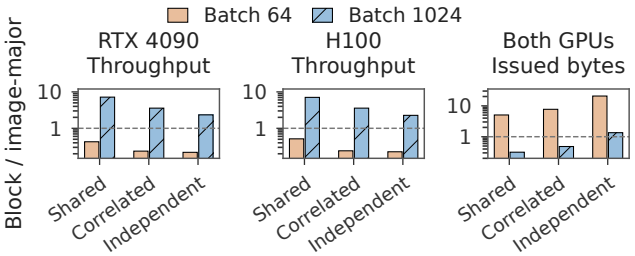


Figure 16: Request population and crop locality on the block-major layout. Large preparation pools amortize vector decoding, while spatial locality improves vector utilization.

H2D traffic from 10.009 to 1.232 GB. End-to-end time drops from 4.557 to 3.386 s on RTX 4090 and from 4.727 to 3.342 s on H100, improving throughput by 1.35× and 1.41×, respectively. For the matched 9.865 GB logical coefficient request, GPU decoding takes only 17.87 and 19.42 ms, while cumulative H2D time falls from 516.07 to 59.63 ms and from 393.89 to 49.22 ms, respectively. Thus, compression substantially reduces data movement and improves throughput without shifting the bottleneck to decompression.

Direct tensor construction. Finally, we examine whether input reduction is preserved after decoding. The second control in Figure 15 compares direct tensor construction from selected decoded coefficients with reconstructing full component grids before gathering model inputs. Both paths read 1.084 GB and incur nearly identical H2D traffic (1.232 vs. 1.234 GB). Yet direct construction reduces runtime from 9.157 to 3.382 s on RTX 4090 and from 9.941 to 3.346 s on H100, improving throughput by 2.71× and 2.97×. GPU residency also drops from 21.69 to 3.69 GiB and from 22.20 to 4.21 GiB, respectively. By avoiding full-grid materialization and subsequent gathering, direct construction nearly triples throughput and reduces GPU residency by about 80%, preserving input reduction through model tensor construction.

Takeaway. These controls show that input reduction must be preserved throughout the pipeline. Spatial selection and compact staging reduce physical data movement, compressed transfer lowers delivery costs, and direct tensor construction avoids re-expanding selected data. These mechanisms jointly eliminate unnecessary work.

6.5 Design Trade-off

6.5.1 When Does the Block-Major Layout Help? Block-major groups corresponding spatial positions across images, making its efficiency dependent on cross-image vector utilization. We vary preparation batch size (64 vs. 1,024 images) and crop locality: shared origins, correlated crops within 32 pixels, and independent crops on a 16-pixel grid. Figure 16 compares block-major against image-major tiled execution, which stores each image's blocks contiguously so that a request decodes only its own images' data. At 64 images, block-major achieves only 0.22–0.51× the throughput of image-major while issuing 5.05–20.72× more bytes: since each vector holds one position across the 1,024 images of a partition, a 64-image batch uses at most 64/1024 ≈ 6% of the rows of every decoded vector, and fewer if it spans several partitions. At 1,024 images, block-major becomes 7.07–7.18×, 3.58×, and 2.27–2.34× faster than image-major for shared, correlated, and independent crops, respectively. Shared crops use every row of each touched vector; even independent crops, at about 25% vector utilization, remain over 2.2× faster despite issuing 1.35× more bytes. Thus, larger preparation pools amortize vector decoding, while crop locality improves utilization.

6.5.2 Sensitivity to Input Read Bandwidth. While the preceding experiments use local storage, large-scale data-center workloads often access data through network-attached storage with constrained delivery bandwidth. For example, Amazon EBS gp3 provides 125 MiB/s baseline throughput, up to 2,000 MiB/s [6]. Such constraints make crop pushdown particularly valuable by reducing data transferred from storage. To evaluate this setting, we emulate aggregate read bandwidth limits of 125, 500, and 2,000 MB/s across all reader threads, with payload pages kept warm. We compare full-loading and compact selective execution using the same MobileNetV2-DCT24 configuration as the spatial ablation. We make two observations. First, at 125 MB/s, selective execution improves throughput by 14.8% on RTX 4090 and 14.6% on H100 for 448-pixel crops. For the more selective 160-pixel crops, improvements reach 96.1% and 78.5%, respectively. Second, as bandwidth increases, throughput improvements reduce to at most 7.7% at 500 MB/s and 5.2% at 2,000 MB/s. This shows that projection pushdown provides greater end-to-end benefits when input read delivery is bandwidth-constrained.

7 Conclusion

We presented FastLens, an image input system that preserves model-driven selection from storage to model input. A cross-image block-major FastLanes layout with dependency-aware projection pushdown turns crop and frequency requests into compact compressed reads; a mixed-dispatch GPU decoder keeps the selected data compressed across PCIe, cutting H2D traffic by 87.7%; and direct tensor construction avoids re-expanding it, nearly tripling throughput over full-grid construction. Against the PyTorch DCT pipeline, FastLens speeds up training by 3.80–4.20× and inference by 6.17–7.95× on average, reaching 99.1% of GPU-resident training throughput while storing ImageNet at 2.12× its JPEG size. Block-major access pays off with large preparation pools and crop locality, and gains shrink where model execution dominates. The broader lesson is that input reduction is only as effective as its weakest stage: selection must be preserved through storage, transfer, decoding, and tensor construction. The code is available at https://anonymous.4open.science/r/FastLens.

References

[1] Daniel J. Abadi, Samuel R. Madden, and Miguel Ferreira. 2006. Integrating Compression and Execution in Column-Oriented Database Systems. In *Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data (SIGMOD '06)*. Association for Computing Machinery, New York, NY, USA, 671–682. doi:10.1145/1142473.1142548

[2] Azim Afroozeh and Peter Boncz. 2023. The FastLanes Compression Layout: Decoding >100 Billion Integers per Second with Scalar Code. *Proceedings of the VLDB Endowment* 16, 9 (2023), 2132–2144. doi:10.14778/3598581.3598587

[3] Azim Afroozeh and Peter Boncz. 2025. The FastLanes File Format. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4629–4643. doi:10.14778/3749646.3749718

[4] Azim Afroozeh, Lotte Felius, and Peter Boncz. 2024. Accelerating GPU Data Processing using FastLanes Compression. In *Proceedings of the 20th International Workshop on Data Management on New Hardware*. Article 8, 11 pages. doi:10.1145/3662010.3663450

[5] N. Ahmed, T. Natarajan, and K.R. Rao. 1974. Discrete Cosine Transform. *IEEE Trans. Comput.* C-23, 1 (1974), 90–93. doi:10.1109/T-C.1974.223784

[6] Amazon Web Services. 2026. Amazon EBS General Purpose SSD Volumes. https://docs.aws.amazon.com/ebs/latest/userguide/general-purpose.html. Accessed: 2026-10.

[7] Michael Andersch, Greg Palmer, Ronny Krashinsky, Nick Stam, Vishal Mehta, Gonzalo Brito, and Sridhar Ramaswamy. 2022. NVIDIA Hopper Architecture In-Depth. NVIDIA Technical Blog. Published March 22, 2022. https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/

[8] Jonghyun Bae, Woohyeon Baek, Tae Jun Ham, and Jae W. Lee. 2022. L3: Accelerator-Friendly Lossless Image Format for High-Resolution, High-Throughput DNN Training. In *Computer Vision – ECCV 2022*, Shai Avidan, Gabriel Brostow, Moustapha Cissé, Giovanni Maria Farinella, and Tal Hassner (Eds.). Springer Nature Switzerland, Cham, 171–188.

[9] Lennart Behme, Saravanan Thirumuruganathan, Alireza Rezaei Mahdiraji, Jorge-Arnulfo Quiané-Ruiz, and Volker Markl. 2023. The Art of Losing to Win: Using Lossy Image Compression to Improve Data Loading in Deep Learning Pipelines. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 936–949. doi:10.1109/ICDE55515.2023.00077

[10] Fa-You Chen, Yun-Jui Hsu, Chia-Hsun Lu, Hong-Han Shuai, Lo-Yao Yeh, and Chih-Ya Shen. 2025. Compressing Deep Neural Networks with Goal-Specific Pruning and Self-Distillation. *ACM Trans. Knowl. Discov. Data* 19, 4, Article 91 (May 2025), 27 pages. doi:10.1145/3721293

[11] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 578–594. https://www.usenix.org/conference/osdi18/presentation/chen

[12] Yann Collet and Murray S. Kucherawy. 2021. Zstandard Compression and the 'application/zstd' Media Type. RFC 8878. doi:10.17487/RFC8878

[13] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. ImageNet: A Large-Scale Hierarchical Image Database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 248–255. doi:10.1109/CVPR.2009.5206848

[14] Samuel Felipe dos Santos and Jurandy Almeida. 2021. Less Is More: Accelerating Faster Neural Networks Straight from JPEG. In *Progress in Pattern Recognition, Image Analysis, Computer Vision, and Applications*, João Manuel R. S. Tavares, João Paulo Papa, and Manuel González Hidalgo (Eds.). Springer International Publishing, Cham, 237–247.

[15] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *9th International Conference on Learning Representations (ICLR 2021)*. Virtual Event, Austria. https://openreview.net/forum?id=YicbFdNTTy

[16] Max Ehrlich and Larry Davis. 2019. Deep Residual Learning in the JPEG Transform Domain. In *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*. 3483–3492. doi:10.1109/ICCV.2019.00358

[17] Wenbin Fang, Bingsheng He, and Qiong Luo. 2010. Database Compression on Graphics Processors. *Proc. VLDB Endow.* 3, 1–2 (2010), 670–680. doi:10.14778/1920841.1920927

[18] Kfir Goldberg, Stav Shapiro, Elad Richardson, and Shai Avidan. 2020. Rethinking FUN: Frequency-Domain Utilization Networks. arXiv:2012.03357 [cs.CV] https://arxiv.org/abs/2012.03357

[19] S. Golomb. 1966. Run-length encodings (Corresp.). *IEEE Transactions on Information Theory* 12, 3 (1966), 399–401. doi:10.1109/TIT.1966.1053907

[20] Google. [n. d.]. Snappy: A Fast Compressor/Decompressor. https://github.com/google/snappy. Accessed October 2026.

[21] Lionel Gueguen, Alex Sergeev, Ben Kadlec, Rosanne Liu, and Jason Yosinski. 2018. Faster Neural Networks Straight from JPEG. In *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett (Eds.), Vol. 31. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2018/file/7af6266cc52234b5aa339b16695f7fc4-Paper.pdf

[22] Sasun Hambardzumyan, Abhinav Tuli, Levon Ghukasyan, Fariz Rahman, Hrant Topchyan, David Isayan, Mark McQuade, Mikayel Harutyunyan, Tatevik Hakobyan, Ivo Stranic, and Davit Buniatyan. 2023. Deep Lake: a Lakehouse for Deep Learning. In *Conference on Innovative Data Systems Research (CIDR)*.

[23] Song Han, Huizi Mao, and William J. Dally. 2016. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. In *4th International Conference on Learning Representations (ICLR 2016), Conference Track Proceedings*. San Juan, Puerto Rico, 1–14. https://arxiv.org/abs/1510.00149

[24] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778. doi:10.1109/CVPR.2016.90

[25] Sven Hepkema, Azim Afroozeh, Charlotte Felius, Peter Boncz, and Stefan Manegold. 2025. G-ALP: Rethinking Light-weight Encodings for GPUs. In *Proceedings of the 21st International Workshop on Data Management on New Hardware*. ACM, Berlin Germany, 1–10. doi:10.1145/3736227.3736242

[26] Alexander Isenko, Ruben Mayer, Jeffrey Jedele, and Hans-Arno Jacobsen. 2022. Where Is My Training Bottleneck? Hidden Trade-Offs in Deep Learning Preprocessing Pipelines. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 1825–1839. doi:10.1145/3514221.3517848

[27] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. TASO: optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles* (Huntsville, Ontario, Canada) *(SOSP '19)*. Association for Computing Machinery, New York, NY, USA, 47–62. doi:10.1145/3341301.3359630

[28] Daniel Kang, Ankit Mathur, Teja Veeramacheneni, Peter Bailis, and Matei Zaharia. 2020. Jointly optimizing preprocessing and inference for DNN-based visual analytics. *Proc. VLDB Endow.* 14, 2 (Oct. 2020), 87–100. doi:10.14778/3425879.3425881

[29] Michael Kuchnik, George Amvrosiadis, and Virginia Smith. 2021. Progressive Compressed Records: Taking a Byte out of Deep Learning Data. *Proc. VLDB Endow.* 14, 11 (2021), 2627–2641. doi:10.14778/3476249.3476308

[30] Michael Kuchnik, Ana Klimovic, Jiří Šimša, Virginia Smith, and George Amvrosiadis. 2022. Plumber: Diagnosing and Removing Performance Bottlenecks in Machine Learning Data Pipelines. In *Proceedings of Machine Learning and Systems (MLSys)*.

[31] Andrew Lavin and Scott Gray. 2016. Fast Algorithms for Convolutional Neural Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 4013–4021. doi:10.1109/CVPR.2016.435

[32] Guillaume Leclerc, Andrew Ilyas, Logan Engstrom, Sung Min Park, Hadi Salman, and Aleksander Mądry. 2023. FFCV: Accelerating Training by Removing Data Bottlenecks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 12011–12020. https://openaccess.thecvf.com/content/CVPR2023/html/Leclerc_FFCV_Accelerating_Training_by_Removing_Data_Bottlenecks_CVPR_2023_paper.html

[33] Gang Liao, Ye Liu, Jianjun Chen, and Daniel J. Abadi. 2025. Bullion: A Column Store for Machine Learning. In *Conference on Innovative Data Systems Research (CIDR)*.

[34] Liyang Liu, Shilong Zhang, Zhanghui Kuang, Aojun Zhou, Jing-Hao Xue, Xinjiang Wang, Yimin Chen, Wenming Yang, Qingmin Liao, and Wayne Zhang. 2021. Group Fisher Pruning for Practical Network Compression. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 7021–7032. https://proceedings.mlr.press/v139/liu21ab.html

[35] Ze Liu, Han Hu, Yutong Lin, Zhuliang Yao, Zhenda Xie, Yixuan Wei, Jia Ning, Yue Cao, Zheng Zhang, Li Dong, Furu Wei, and Baining Guo. 2022. Swin Transformer V2: Scaling Up Capacity and Resolution. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 11999–12009. doi:10.1109/CVPR52688.2022.01170

[36] Jayashree Mohan, Amar Phanishayee, Ashish Raniwala, and Vijay Chidambaram. 2021. Analyzing and Mitigating Data Stalls in DNN Training. *Proc. VLDB Endow.* 14, 5 (2021), 771–784. doi:10.14778/3446095.3446100

[37] G. M. Morton. 1966. *A Computer Oriented Geodetic Data Base and a New Technique in File Sequencing*. Technical Report. IBM Ltd., Ottawa, Ontario, Canada.

[38] Derek G. Murray, Jiří Šimša, Ana Klimovic, and Ihor Indyk. 2021. tf.data: a machine learning data processing framework. *Proc. VLDB Endow.* 14, 12 (July 2021), 2945–2958. doi:10.14778/3476311.3476374

[39] NVIDIA. [n. d.]. NVIDIA H100 GPU. Product specifications. Accessed October 8, 2026. https://www.nvidia.com/en-us/data-center/h100/

[40] NVIDIA. 2026. DALI: Region-of-Interest Decoding and Decoder Pipeline Examples. Version 1.50.0 examples; accessed September 14, 2026. https://docs.nvidia.com/deeplearning/dali/archives/dali_1_50_0/user-guide/examples/image_processing/decoder_examples.html

[41] NVIDIA. 2026. DALI v1.50.0 FileLabelLoader Source. Versioned source; accessed September 14, 2026. https://raw.githubusercontent.com/NVIDIA/DALI/v1.50.0/dali/operators/reader/loader/file_label_loader.cc

[42] NVIDIA. 2026. NVIDIA Data Loading Library (DALI). Accessed September 12, 2026. https://developer.nvidia.com/dali

[43] Jack A. Orenstein. 1986. Spatial query processing in an object-oriented database system. In *Proceedings of the 1986 ACM SIGMOD International Conference on Management of Data* (Washington, D.C., USA) *(SIGMOD '86)*. Association for Computing Machinery, New York, NY, USA, 326–336. doi:10.1145/16894.16886

[44] Weston Pace, Chang She, Lei Xu, Will Jones, Albert Lockett, Jun Wang, and Raunak Shah. 2025. Lance: Efficient Random Access in Columnar Storage through Adaptive Structural Encodings. arXiv:2504.15247

[45] Jeongsoo Park and Justin Johnson. 2023. RGB No More: Minimally-Decoded JPEG Vision Transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 22334–22346. https://openaccess.thecvf.com/content/CVPR2023/html/Park_RGB_No_More_Minimally-Decoded_JPEG_Vision_Transformers_CVPR_2023_paper.html

[46] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. *PyTorch: an imperative style, high-performance deep learning library*. Curran Associates Inc., Red Hook, NY, USA.

[47] Zhongwei Qiu, Huan Yang, Jianlong Fu, and Dongmei Fu. 2022. Learning Spatiotemporal Frequency-Transformer for Compressed Video Super-Resolution. In *Computer Vision – ECCV 2022*, Shai Avidan, Gabriel Brostow, Moustapha Cissé, Giovanni Maria Farinella, and Tal Hassner (Eds.). Springer Nature Switzerland, Cham, 257–273.

[48] K.R. Rao and P.C. Yip. 1990. *Discrete Cosine Transform: Algorithms, Advantages, Applications*. Elsevier Science & Technology Books. https://books.google.com/books?id=dSoaogEACAAJ

[49] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. 2015. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision* 115, 3 (Dec. 2015), 211–252. doi:10.1007/s11263-015-0816-y

[50] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. MobileNetV2: Inverted Residuals and Linear Bottlenecks. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 4510–4520. doi:10.1109/CVPR.2018.00474

[51] Anil Shanbhag, Bobbi W. Yogatama, Xiangyao Yu, and Samuel Madden. 2022. Tile-based Lightweight Integer Compression in GPU. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD '22)*. 1390–1403. doi:10.1145/3514221.3526132

[52] Utku Sirin and Stratos Idreos. 2024. The Image Calculator: 10x Faster Image-AI Inference by Replacing JPEG with Self-designing Storage Format. *Proceedings of the ACM on Management of Data* 2, 1, Article 52 (2024), 31 pages. doi:10.1145/3639307

[53] Utku Sirin and Stratos Idreos. 2025. Data Storage and Management for Image AI Pipelines. In *Companion of the 2025 International Conference on Management of Data*. 855–863. doi:10.1145/3722212.3725640

[54] Utku Sirin, Victoria Kauffman, Aadit Saluja, Florian Klein, Jeremy Hsu, and Stratos Idreos. 2025. Frequency-Store: Scaling Image AI by A Column-Store for Images. In *Conference on Innovative Data Systems Research (CIDR)*. https://vldb.org/cidrdb/papers/2025/p27-sirin.pdf

[55] Mingxing Tan and Quoc Le. 2019. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *Proceedings of the 36th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 97)*, Kamalika Chaudhuri and Ruslan Salakhutdinov (Eds.). PMLR, 6105–6114. https://proceedings.mlr.press/v97/tan19a.html

[56] Gregory K. Wallace. 1991. The JPEG still picture compression standard. *Commun. ACM* 34, 4 (April 1991), 30–44. doi:10.1145/103085.103089

[57] Welch. 1984. A Technique for High-Performance Data Compression. *Computer* 17, 6 (1984), 8–19. doi:10.1109/MC.1984.1659158

[58] Kai Xu, Minghai Qin, Fei Sun, Yuhao Wang, Yen-Kuang Chen, and Fengbo Ren. 2020. Learning in the Frequency Domain. *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2020), 1737–1746. https://api.semanticscholar.org/CorpusID:211572645

[59] Lijie Xu, Shuang Qiu, Binhang Yuan, Jiawei Jiang, Cedric Renggli, Shaoduo Gan, Kaan Kara, Guoliang Li, Ji Liu, Wentao Wu, Jieping Ye, and Ce Zhang. 2022. In-Database Machine Learning with CorgiPile: Stochastic Gradient Descent without Full Data Shuffle. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD '22)*. Association for Computing Machinery, 1286–1300. doi:10.1145/3514221.3526150

[60] Jiaqi Zhao, Yunfei Yi, Yunyin Shi, and Peng Tang. 2025. Hardware-Aware Automated Quantization for Balancing Latency and Accuracy. In *2025 IEEE International Conference on Pattern Recognition, Machine Vision and Artificial Intelligence (PRMVAI)*. 1–4. doi:10.1109/PRMVAI65741.2025.11108556

[61] Mark Zhao, Emanuel Adamiak, and Christos Kozyrakis. 2024. cedar: Optimized and Unified Machine Learning Input Data Pipelines. *Proc. VLDB Endow.* 18, 2 (Oct. 2024), 488–502. doi:10.14778/3705829.3705861

[62] Marcin Zukowski, Sándor Héman, Niels Nes, and Peter Boncz. 2006. Super-Scalar RAM-CPU Cache Compression. In *Proceedings of the 22nd International Conference on Data Engineering (ICDE '06)*. IEEE Computer Society, Washington, DC, USA, 59. doi:10.1109/ICDE.2006.150